

Program 1:

Build a Docker Container from a custom Dockerfile

31/10/2025

Project Directory:

```
prog1
|- app.py
|- requirements.txt
|- Dockerfile
```

Program Code and steps for execution:

Step 1: Move to the directory where we work for DevOps lab. Then create a project directory with the name prog1

```
> mkdir prog1
```

```
> cd prog1/
```

```
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~$ cd devops_lab/
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab$ mkdir prog1
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab$ cd prog1
```

Step 2: Inside the prog1 folder there will be 3 files

1. requirements.txt 2. app.py 3. Dockerfile

All the above files contains the code which is in below pictures respectively

```
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog1$ sudo nano requirements.txt
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog1$ cat requirements.txt
flask
```

```
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog1$ sudo nano app.py
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog1$ cat app.py
from flask import Flask

app=Flask(__name__)

@app.route("/")
def hello():
    return "Hello, Docker!!!"

if __name__=="__main__":
    app.run(host="0.0.0.0", port=5000)
```

```
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog1$ sudo nano Dockerfile
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog1$ cat Dockerfile
### Dockerfile
```

```
# This is the Dockerfile which contains the information to build the docker container images
```

```
# Use the slim python as a base image
FROM python:3.9-slim
```

```
# Set the working directory for the container
WORKDIR /app
```

```
# Copy the text/content from the requirements.txt and install all the dependencies
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
```

```
# Copy the application code
COPY . .
```

```
# Set the flask environment variables
ENV FLASK_APP=app.py
ENV FLASK_RUN_HOST=0.0.0.0
ENV FLASK_RUN_PORT=5000
```

```
# Expose the port that flask will run an app on
EXPOSE 5000
```

```
# Define the commands to run the flask application
CMD ["flask", "run"]
```

Step 3: The final Project directory looks like the below

```
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog1$ ls
app.py  Dockerfile  requirements.txt
```

Step 4: Build the docker image from the Dockerfile of prog1 using the command,

> docker build -t prog1 .

```
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog1$ docker build -t prog1 .
[+] Building 2.3s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 710B
=> [internal] load metadata for docker.io/library/python:3.9-slim
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aab1acda1e13cf1731b1b
=> => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aab1acda1e13cf1731b1b
=> [internal] load build context
=> => transferring context: 269B
=> CACHED [2/5] WORKDIR /app
=> CACHED [3/5] COPY requirements.txt .
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt
=> [5/5] COPY . .
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:fdcb9ac48df4e541b1800968ec4077e4c8fb81b468b32c90d4730e8254e01e84
=> => exporting config sha256:6de5de28faba26fb5b8c79b981e16cd4a02a9a35e370dc2d94c93b10a56c388a
=> => exporting attestation manifest sha256:52188d73cc8c060abf971e9401b5fb4565d20705aa402fd11afacdeaf835a2f0
=> => exporting manifest list sha256:d10a716af680c988678c93a054841ed329105cd5614b2c6f6b959bcf6438019b
=> => naming to docker.io/library/prog1:latest
=> => unpacking to docker.io/library/prog1:latest
```

Step 5: Run the image which has been built by using the command,

> docker run -d -p 5000:5000 --name flask-container prog1

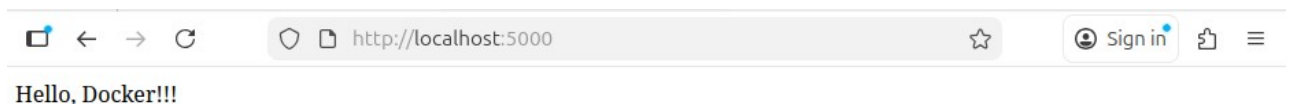
Step 6: Check for running containers

```
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog1$ docker run -d -p 5000:5000 --name flask-container prog1
c730a91d8013f5f7418d1461fba12635c3e1a0a30e7b0dcf2f6ff6302c4276fa
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog1$ docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
c730a91d8013   prog1          "flask run"             6 seconds ago Up 5 seconds   0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp   flask-container
```

Step 7: Open any browser in the local machine and search for,

> <http://localhost:5000/>

The output will be as show below



Step 8: Stop and remove all the running container, also remove the created image by using the command,

> docker container stop <4_characters_of_container_id>

> docker container rm <4_characters_of_container_id>

> docker rmi <4_characters_of_image_id>

```
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog1$ docker container stop c730
c730
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog1$ docker container rm c730
c730
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog1$ docker images
IMAGE          ID                  DISK USAGE  CONTENT SIZE  EXTRA
prog1:latest   d10a716af680       200MB       48.8MB
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog1$ docker rmi prog1:latest
Untagged: prog1:latest
Deleted: sha256:d10a716af680c988678c93a054841ed329105cd5614b2c6f6b959bcf6438019b_
```